Nº ALUNO: ***NOME:***

(0.5v)

1. **As linguagens interpretadas…**
   - **A.** necessitam de uma compilação prévia antes de serem executadas.
   - **B.** não são consideradas linguagens de programação.
   - **C.** podem conter erros de sintaxe e ainda assim executar parte do seu código.
   - **D.** são usadas apenas no desenvolvimento *front-end.*

(0.5v)

2. **Quando uma linguagem de programação tem *dynamic typing,***
   - **A.** a verificação do tipo da variável é realizada quando o programa é compilado.
   - **B.** significa que é necessário declarar uma variável antes de ser atribuída.
   - **C.** significa que tem capacidade de atribuir diferentes tipos à mesma variável, sendo a verificação do tipo realizada em tempo de execução.
   - **D.** significa que o tipo de uma variável não pode variar ao longo do seu tempo de vida.

(0.75v)

3. **Considere o seguinte bloco de código JavaScript e selecione a afirmação correta, considerando que no documento html existe um elemento com id=btn.**

```
document.queryselector("#btn").addEventListener("click",function()
                                        { console.log("click!"); });
```

   - **A.** O bloco de código não apresenta qualquer função *callback*.
   - **B.** O bloco de código apresenta uma função *callback*, definida de forma anónima.
   - **C.** Qualquer declaração de eventos com recurso ao *addEventListener* não usa funções *callback*.
   - **D.** O bloco de código apresenta a função callback cujo nome é *click*.

(0.5v)

4. **No contexto da propagação de eventos em JavaScript, a fase de *event capturing***
   - **A.** inicia com a captação do evento do elemento mais externo e, de seguida, propagando-o para o elemento mais interno.
   - **B.** inicia com a captação do evento do elemento mais interno e, de seguida, propagando-o para o elemento mais externo.
   - **C.** é o momento em que captação do evento inicia.
   - **D.** é o momento em que captação do evento termina.

(0.5v)

5. **No contexto de *Multi Page Applications* (MPA) e *Single Page Applications* (SPA), o React é utilizado**
   - **A.** apenas na implementação de SPA.
   - **B.** apenas na implementação de MPA.
   - **C.** bastante na implementação de SPA podendo também ser usado em partes de MPA.
   - **D.** como uma alternativa ao HTML na implementação de MPA.

(0.5v)

6. **Em JavaScript um objeto literal (literal notation):**
   - **A.** pode ser criado sem propriedades nem métodos, podendo estes elementos ser adicionados posteriormente, mesmo que tenha sido declarado com const.
   - **B.** pode ser criado sem propriedades nem métodos, não podendo estes elementos ser adicionados posteriormente caso tenha sido declarado com const.
   - **C.** tem de ser declarado com, pelo menos, uma propriedade aquando da sua declaração.
   - **D.** deve incorporar todos as propriedades e métodos quando declarado como const.

(0.75v)

7. **Considerando a seguinte declaração de objeto, qual das opções permite desestruturar o objeto em JavaScript?**

```
const exame = { epoca:'Normal', uc:'LS', ano:1}
```

   - **A.** `const {epoca,uc,ano}={exame.epoca,exame.uc,exame.ano};`
   - **B.** `const [epoca,uc,ano]=exame;`
   - **C.** `const {epoca,uc,ano}=exame;`
   - **D.** `Nenhuma das opções apresentadas se encontra correta.`

(0.75v)

8. Considere os seguintes blocos de código e selecione a afirmação correta

```
const exame = x => 28;
console.log(exame(10,10,8))
```

```
console.log(exame(28))
const exame = x => x;
```

A. Os dois blocos apresentam 28.
B. Os dois blocos apresentam erros de código.
C. O bloco à esquerda apresenta 28 e o bloco da direita erro.
D. O bloco à esquerda apresenta erro e o bloco da direita 28.

(0.5v)

9. Qual o resultado de `33 === 1+2+'3'` em JavaScript
A. True
B. False
C. *null*
D. *Erro*

(1v)

10. Considerando o seguinte código, qual o valor de b?

```
let a = [1, 4];
let b = [...a, ...a];
a.push(5);
```

A. [1, 4, 1, 4, 5]
B. [1, 2, 3, 4, 1, 2, 3, 4]
C. [1, 4,5]
D. [1, 4, 1, 4]

(0.5v)

11. No contexto da implementação de classes em JavaScript, a função *super* é usada para
A. criar super variáveis, que estão disponíveis em todos os escopos de classe.
B. invocar o construtor da classe pai.
C. inicializar propriedades e valores, e por isso chamado antes do construtor da classe.
D. permitir o uso do 'this' nas classes.

(0.5v)

12. O conceito de *lifecycle* (ciclo de vida) de um componente em React refere-se às diferentes fases do componente
A. permitindo usar métodos, tais como o *componentDidMount()* e *componentDidUpdate()* existentes em componentes funcionais, que permitem implementar o comportamento desejado.
B. permitindo usar os métodos *componentDidMount()* e o *componentDidUpdate(),* existentes nos componentes de class, sendo obrigatórios a sua implementação.
C. permitindo usar métodos como o *useEffect()* em componentes funcionais, para implementar o comportamento desejado.
D. não existindo, no entanto, esse conceito sempre que se recorre aos Hooks.

(0.5v)

13. Considerando apenas os conceitos de props e estado, qual das afirmações é correta no contexto do React?
A. As *props* podem ser passadas do componente filho para o componente pai.
B. Funções callback permitem passar o estado de um componente filho para componente pai.
C. O estado de um componente passa de forma automática de pai para filho, sem ser necessário recorrer às props.
D. O *estado* e as *props* podem ser passadas do componente filho para o componente pai.

(1v)

14. Qual o conteúdo da variável y no final do seguinte bloco de código JavaScript?

```
const x = [1,2,3];
const y=x;
x[y.length]=4;
```

A. [1, 2, 4]
B. [1, 2, 3]
C. [1, 2, 3, 4]
D. Nenhuma das opções se encontra correta

Nº ALUNO: ***NOME:***

(0.5v)

15. Em React, como é possível passar um *event handler* para um componente?

A. em react, não faz sentido passar *event handlers* para um componente.
B. Não é possível passar *event handlers* para componentes, estes devem ser definidos sempre em cada componente.
C. Obrigatoriamente usando um estado.
D. Através de uma *prop*.

(0.5v)

16. Qual o objetivo do JSX no contexto de uma aplicação React?

A. para especificar um bloco de código JavaScript.
B. permite agrupar um conjunto de elementos, sem ser necessário, adicionar um novo elemento ao DOM.
C. permite combinar código JavaScript com HTML.
D. para estilizar elementos do DOM através do React.

(0.5v)

17. Qual o objetivo da propriedade key num elemento JSX dentro de um ciclo?

A. Especificar estilos para cada elemento na lista.
B. Não é necessário utilizar a propriedade key em elementos JSX.
C. Identificar unicamente cada elemento na lista.
D. Passar dados do componente pai para um componente filho.

(1v)

18. Qual o output do seguinte código?

```
function doSomething() {
	const l = ["Amarelo", "Azul", "Vermelho"];
	l.forEach((c) => {
		c += " escuro";
	});
	console.log(l);
}
doSomething();
```

A. ["Amarelo escuro", "Azul escuro", "Vermelho escuro"]
B. ["Amarelo", "Azul", "Vermelho", " escuro"]
C. ["Amarelo", "Azul", "Vermelho"]
D. Erro de código

(0.5v)

19. Qual dos seguintes métodos de criação de objetos necessita do uso do operador new para instanciar o objeto?

A. Quando se recorre a *Construtor Functions ou Classes.*
B. Quando se recorre a *Factory Functions ou Construtor Functions*.
C. Quando se recorre a *Factory Functions ou Classes.*
D. Todas as opções estão corretas.

(1v)

20. Considere a seguinte função de expressão:

```
let exame = function(x,y) {
     return x + y;
}
```

Qual das seguintes opções implementa a *arrow function* com o mesmo comportamento que a função de expressão apresentada?

A. `let exame(x,y) => () = x + y;`
B. `let exame(x,y) = () => x + y;`
C. `let exame = (x,y) => { x + y };`
D. `let exame = (x,y) => { return x + y; }`

(0.5v)

21. Qual a afirmação correta tendo em consideração o conceito de *hoisting* do JavaScript?

- **A.** Permite a verificação de erros mais rigorosa durante a execução do JavaScript.
- **B.** Processo de converter código JavaScript para uma versão mais antiga de forma a ser possível o mesmo ser interpretado pelos diferentes browsers.
- **C.** Comportamento em JavaScript em que move as declarações para o topo do *scope* presente.
- **D.** Todas as afirmações apresentadas estão corretas.

(1v)

22. Qual o output do seguinte código?

```
function doSomething(a, ...b) {
  return b.map((c) => c * a);
}
let arr = doSomething(5,2,1,6);
console.log(arr);
```

- **A.** [10, 2, 1, 6]
- **B.** [10, 5, 30]
- **C.** undefined
- **D.** [25, 10, 5, 30]

(1v)

23. Qual o output do seguinte código (não tenha em consideração as mudanças de linha)?

```
let x="exame";
function lsFunction(x="normal")  {
  console.log(`${x}`)
  x="LS"
}
lsFunction();
console.log(`${x}`)
```

- **A.** normal exame
- **B.** exame exame
- **C.** *normal LS*
- **D.** *LS LS*

(0.5v)

24. O *spread operator* pode ser utilizado para passar props para um componente no React?

- **A.** Sim, o *spread operator* pode ser utilizado para passar todas as props de um objeto para um componente.
- **B.** Não, o *spread operator* não pode ser utilizado para passar props para componentes no React.
- **C.** Sim, mas apenas para props específicas, não para todas as props do objeto.
- **D.** Sim, mas somente se o componente for um componente de classe.

(0. 5v)

25. Considere o seguinte bloco de código:

```
function Exame(disciplina, epoca="Normal") {
     this.epoca = epoca;
     this.disciplina = disciplina;
}
Exame.prototype.getInfo = function () {
     return `Exame ${this.epoca} de ${this.disciplina}`;
}
```

Qual das seguintes alíneas permite criar uma instância do objeto Exame, época Normal e disciplina "LS"?

- **A.** const o1 = Exame('LS', 'Normal');
- **B.** const o1 = new Exame('LS');
- **C.** Nenhuma das opções apresentadas está correta.
- **D.** const o1 = Exame();
  o1.epoca ='Normal';
  o1.disciplina='LS';

Nº ALUNO: ***NOME:***

(0.5v)

26. Em relação ainda à alínea anterior, considere que a variável o1 é uma instância do objeto Exame, criada corretamente, qual o output de console.log(o1.getInfo()) ?

A. Exame Normal de LS
B. Exame *undefined* de *undefined*
C. Erro pois não reconhece a função getInfo
D. Exame Normal de *undefined*

(0.5v)

27. O código apresentado na alínea 25) pode ser substituído pela seguinte declaração de classe?

```
class Exame {
  constructor(disciplina, epoca = "Normal") {
    this.epoca = epoca;
    this.disciplina = disciplina;
  }
  getInfo() {
    return `Exame ${this.epoca} de ${this.disciplina}`;
  } }
```

A. Não, pois o método getInfo() não deve ser um método da classe.
B. Sim.
C. Não, pois são declarações completamente diferentes.
D. Sim, se todos os ***this.*** existentes na class forem removidas.

(0.75v)

28. Considere uma aplicação React contendo a estrutura de diretórios completa e módulos necessários instalados, contendo os seguintes ficheiros de componentes, localizados na pasta src:

Ficheiro App.js

```
import React from "react";
import {useEffect} from "react";
import {useState} from "react";
export default function App() {
  const [exameS, setExameS] =
                    useState(false);

const handle = () => {
    setExameS((stat) => !stat);
};
useEffect(() => {
    console.log("Passou1");
  }, [exameS]);

useEffect(() => {
    console.log("Passou2");
    return () => {
      console.log("Passou 3");
    };
}, []);
return (<span>
  <Info exameS={exameS} />
  <ExameComp2 exameS={exameS}
                h1={handle}/>
    </ span >);
}
```

Ficheiro exame.js

```
import React from "react";

export default class ExameComp1 extends
React.Component {
  render() {
    return (<>{this.props.exameS ?
              "Começou!": "Aguardar..."}
             </>
            );
    }
}

export function ExameComp2({exameS, h1})
{
  let but;
  if (!exameS)
    but = <span onClick={h1}>
          Clique para iniciar </span>;
  else
    but=<span onClick={h1}>
         Terminar</span>;

  return <>{but}</>;
}
```

Considerando que não existem erros de código na implementação dos componentes existentes no ficheiro exame.js, de que forma pode especificar o import no ficheiro App.js para que este não apresente qualquer erro?

A.
```
import { ExameComp1, ExameComp2 } from './exame.js'
```
B.
```
import Info, { ExameComp2 } from './exame.js'
```
C.
```
import  {Info}, ExameComp2 from './exame.js'
```
D.
```
import ExameComp1 from './exame.js'
import ExameComp2 from './exame.js'
```

(0,5v)

29. Que tipo de componente React é o ExameComp2?

A. componente funcional, com estado;
B. componente funcional, sem estado;
C. componente de *class*, sem estado;
D. componente de *class*, com estado;

(0,5v)

30. Qual o output apresentado no browser, considerando que App.js é o componente principal a ser renderizado.

A. Começou! Clique para iniciar
B. Aguardar…. Terminar
C. Aguardar…. Clique para iniciar
D. Começou! Terminar

(0,5v)

31. Quando se clica no texto "Terminar" ou "Clique para iniciar", o que acontece de seguida ?

A. O texto "Terminar" muda para "Clique para iniciar" bem como o texto "Começou!" muda para "Aguardar…." e vice-versa.
B. Não acontece nada.
C. Apenas o texto "Terminar" muda para "Clique para iniciar" e vice-versa.
D. Apenas o texto "Começou!" muda para "Aguardar…." e vice-versa.

(0,5v)

32. Ao clicar no texto "Terminar" ou "Clique para iniciar", qual *output* é apresentado na consola?

A. Passou2 e Passou3
B. Não aparece nada.
C. Passou1 e Passou2
D. Passou1